

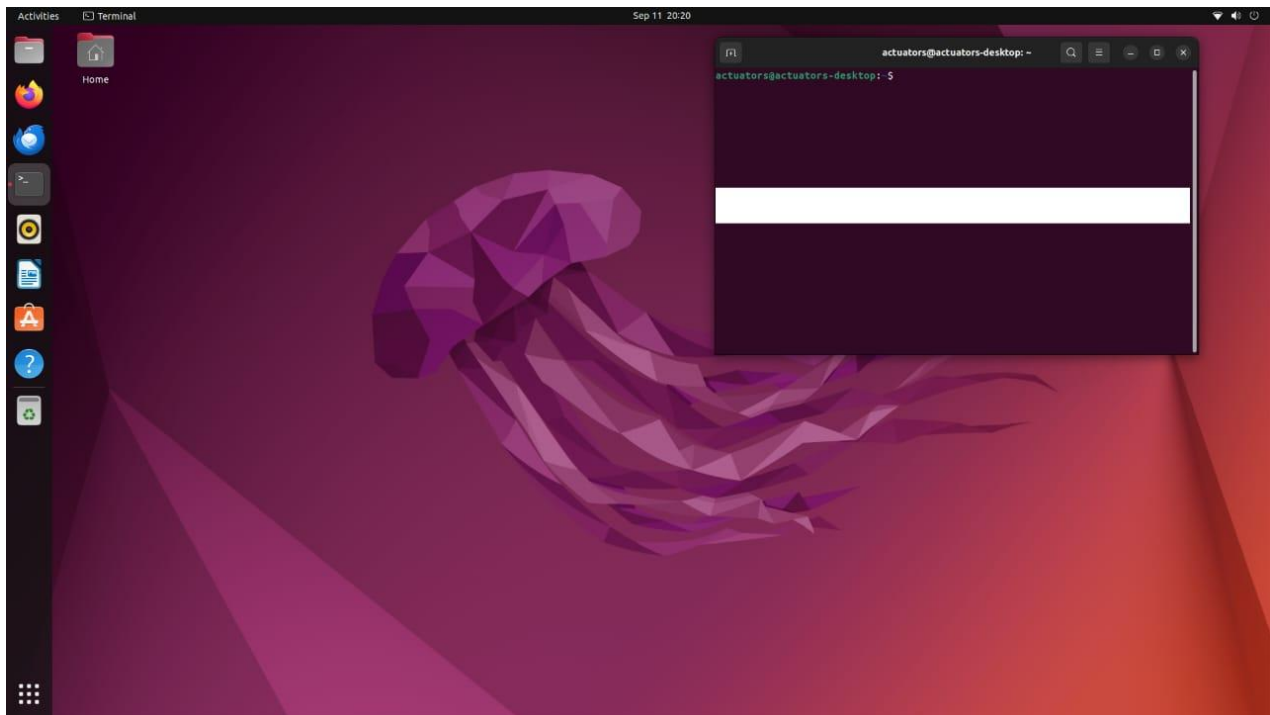
Gazebo Simulation

1. ROS2 Installation on Raspberry Pi

We began our development by installing ROS2 on Ubuntu, which was running on a Raspberry Pi with a 16GB SD card. The Raspberry Pi was connected to a monitor via HDMI and equipped with standard I/O peripherals (keyboard and mouse) to interact with the Ubuntu interface.

Following the guidance provided by Mr. Mukesh and using the official documentation, we successfully installed ROS2.

Reference: <https://docs.ros.org/en/dashing/Installation/Ubuntu-Install-Debians.html>



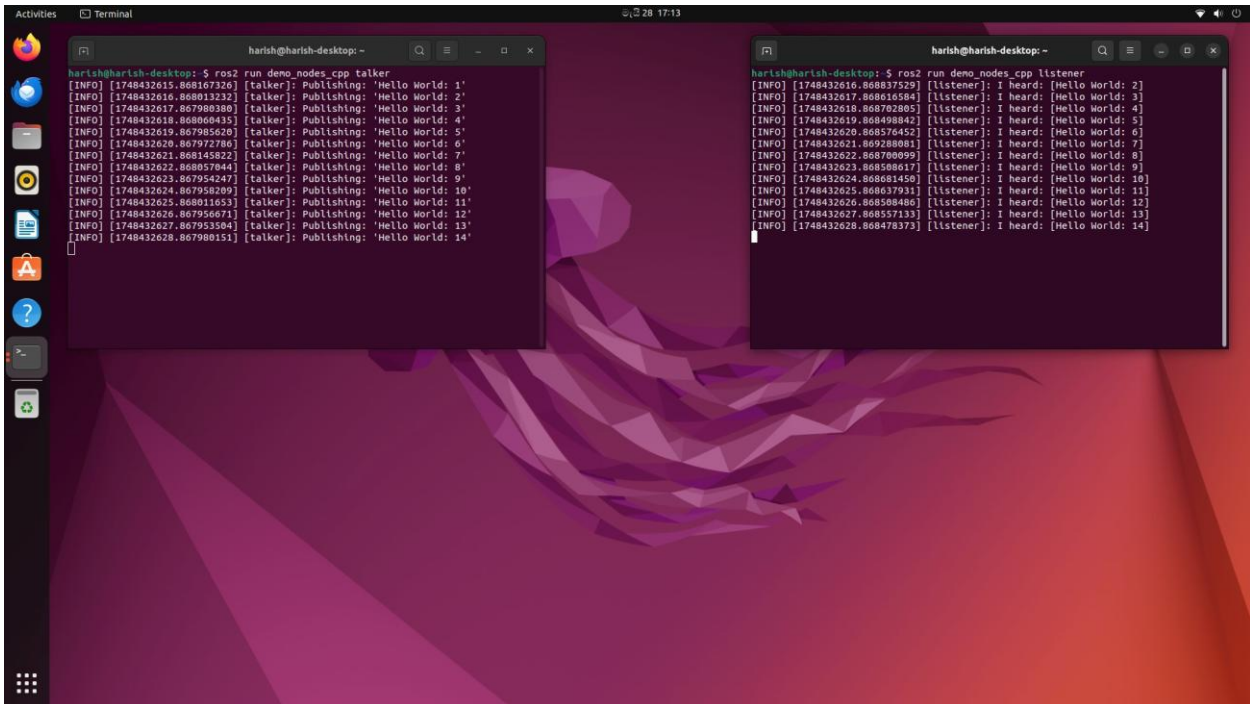
2. Verifying ROS2 Installation

To verify that ROS2 was installed correctly, we used the following commands in two separate terminals:

```
bash CopyEdit
ros2 run demo_nodes_cpp talker ros2 run
demo_nodes_py listener
```

The “talker” node publishes messages, while the “listener” node subscribes to them. This verified that ROS2’s publish–subscribe mechanism was functioning as expected.

Note : Due to the unavailability of the MicroSD in our team, we joined hands with two of the members of the “Cardboard Drone” Team starting from this step.



The image shows two terminal windows on a Linux desktop. The left window, titled 'harish@harish-desktop: ~', shows the output of 'run_demo_nodes_cpp talker'. It displays a series of messages where the 'talker' publishes 'Hello World' from 1 to 14. The right window, also titled 'harish@harish-desktop: ~', shows the output of 'run_demo_nodes_cpp listener'. It displays a series of messages where the 'listener' receives 'Hello World' from 2 to 14. Both windows show timestamps and IP addresses for each message.

```
harish@harish-desktop: ~  
$ ros2 run demo_nodes_cpp talker  
[INFO] [1748432615.868107326] [talker]: Publishing: 'Hello World: 1'  
[INFO] [1748432616.868013232] [talker]: Publishing: 'Hello World: 2'  
[INFO] [1748432617.867980380] [talker]: Publishing: 'Hello World: 3'  
[INFO] [1748432618.868004935] [talker]: Publishing: 'Hello World: 4'  
[INFO] [1748432619.867985620] [talker]: Publishing: 'Hello World: 5'  
[INFO] [1748432620.867972786] [talker]: Publishing: 'Hello World: 6'  
[INFO] [1748432621.868145822] [talker]: Publishing: 'Hello World: 7'  
[INFO] [1748432622.868057044] [talker]: Publishing: 'Hello World: 8'  
[INFO] [1748432623.867954247] [talker]: Publishing: 'Hello World: 9'  
[INFO] [1748432624.867958209] [talker]: Publishing: 'Hello World: 10'  
[INFO] [1748432625.868011653] [talker]: Publishing: 'Hello World: 11'  
[INFO] [1748432626.867956671] [talker]: Publishing: 'Hello World: 12'  
[INFO] [1748432627.867953504] [talker]: Publishing: 'Hello World: 13'  
[INFO] [1748432628.867980151] [talker]: Publishing: 'Hello World: 14'
```

```
harish@harish-desktop: ~  
$ ros2 run demo_nodes_cpp listener  
[INFO] [1748432616.86837529] [listener]: I heard: [Hello World: 2]  
[INFO] [1748432617.868616584] [listener]: I heard: [Hello World: 3]  
[INFO] [1748432618.868702805] [listener]: I heard: [Hello World: 4]  
[INFO] [1748432619.868498842] [listener]: I heard: [Hello World: 5]  
[INFO] [1748432620.868576452] [listener]: I heard: [Hello World: 6]  
[INFO] [1748432621.869288081] [listener]: I heard: [Hello World: 7]  
[INFO] [1748432622.868700099] [listener]: I heard: [Hello World: 8]  
[INFO] [1748432623.868506637] [listener]: I heard: [Hello World: 9]  
[INFO] [1748432624.868681450] [listener]: I heard: [Hello World: 10]  
[INFO] [1748432625.868637931] [listener]: I heard: [Hello World: 11]  
[INFO] [1748432626.868580408] [listener]: I heard: [Hello World: 12]  
[INFO] [1748432627.868557133] [listener]: I heard: [Hello World: 13]  
[INFO] [1748432628.868478373] [listener]: I heard: [Hello World: 14]
```

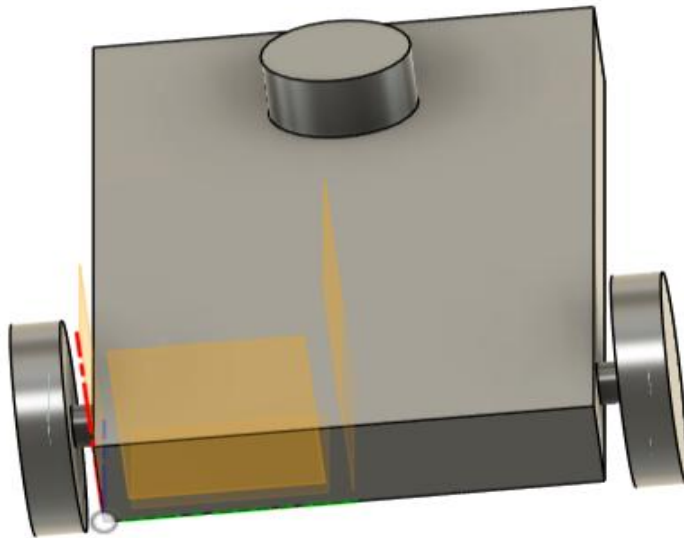
3. Designing the Robot in Fusion 360

We designed the robot model using Fusion 360. However, Fusion 360 does not natively support exporting files in URDF format. To solve this, we downloaded a URDF exporter script from a GitHub repository and integrated it into Fusion 360 via the “Scripts and Add-ins” feature.

Following this integration, we successfully exported our design in a format compatible with ROS.

Reference video:  [Fusion 360 to URDF | Simple Diff Drive Robot Modelling | ROS | Mappin...](#)

Design: https://drive.google.com/drive/folders/1YRwYymGaz_z6zvXFzdWMAjwJrOvZUykm?usp=drive_link



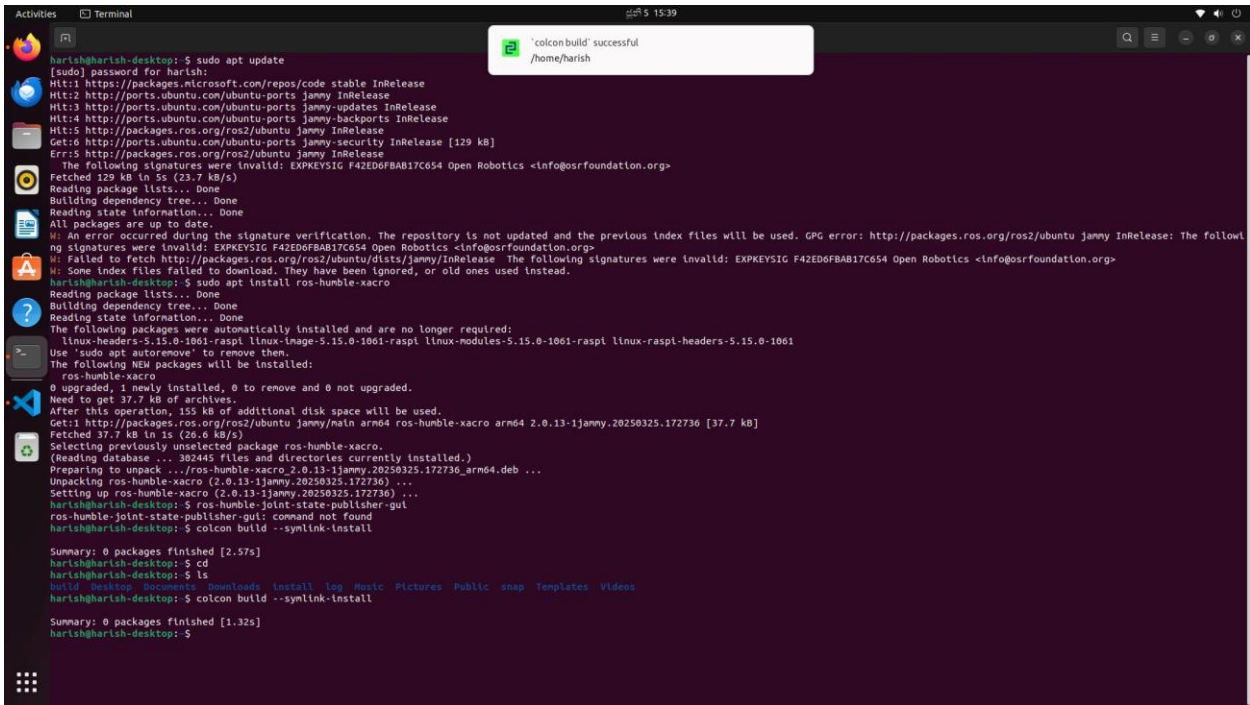
4. Installing COLCON

COLCON is a command-line build tool for ROS2 that replaces catkin in ROS1. It enables us to build packages, manage dependencies, and handle complex ROS2 workspaces.

We installed it using the following commands:

```
bash CopyEdit sudo  
apt update  
sudo apt install python3-colcon-common-extensions
```

Reference: <https://colcon.readthedocs.io/en/released/user/installation.html>



```
harish@harish-desktop:~$ sudo apt update
[sudo] password for harish:
Hit:1 https://packages.microsoft.com/repos/code stable InRelease
Hit:2 http://ports.ubuntu.com/ubuntu-ports jammy InRelease
Hit:3 http://ports.ubuntu.com/ubuntu-ports jammy-updates InRelease
Hit:4 http://ports.ubuntu.com/ubuntu-ports jammy-backports InRelease
Hit:5 http://packages.ros.org/ros2/ubuntu jammy InRelease
Get:6 http://ports.ubuntu.com/ubuntu-ports jammy-security InRelease [129 kB]
Err:5 http://packages.ros.org/ros2/ubuntu jammy InRelease
  The following signatures were invalid: EXPKEYSIG F42ED6FBAB17C654 Open Robotics <info@osrfoundation.org>
Fetched 129 kB in 5s (23.7 kB/s)
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
All packages are up to date.
W: An error occurred during the signature verification. The repository is not updated and the previous index files will be used. GPG error: http://packages.ros.org/ros2/ubuntu jammy InRelease: The following signatures were invalid: EXPKEYSIG F42ED6FBAB17C654 Open Robotics <info@osrfoundation.org>
W: Failed to fetch http://packages.ros.org/ros2/ubuntu/dists/jammy/InRelease The following signatures were invalid: EXPKEYSIG F42ED6FBAB17C654 Open Robotics <info@osrfoundation.org>
W: Some index files failed to download. They have been ignored, or old ones used instead.
harish@harish-desktop:~$ sudo apt install ros-humble-xacro
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
The following packages were automatically installed and are no longer required:
  linux-headers-5.15.0-1061-raspi linux-image-5.15.0-1061-raspi linux-modules-5.15.0-1061-raspi linux-raspi-headers-5.15.0-1061
Use 'sudo apt autoremove' to remove them.
The following NEW packages will be installed:
  ros-humble-xacro
0 upgraded, 1 newly installed, 0 to remove and 0 not upgraded.
Need to get 37.7 kB of archives.
After this operation, 155 kB of additional disk space will be used.
Get:1 http://packages.ros.org/ros2/ubuntu jammy/main arm64 ros-humble-xacro arm64 2.0.13-1jammy.20250325.172736 [37.7 kB]
Fetched 37.7 kB in 1s (26.6 kB/s)
Selecting previously unselected package ros-humble-xacro.
(Reading database ... 302445 files and directories currently installed.)
Preparing to unpack .../ros-humble-xacro_2.0.13-1jammy.20250325.172736_arm64.deb ...
Unpacking ros-humble-xacro (2.0.13-1jammy.20250325.172736) ...
Setting up ros-humble-xacro (2.0.13-1jammy.20250325.172736) ...
harish@harish-desktop:~$ ros-humble-joint-state-publisher-gui
ros-humble-joint-state-publisher-gui: command not found
harish@harish-desktop:~$ colcon build --symlink-install

Summary: 0 packages finished [2.57s]
harish@harish-desktop:~$ cd
harish@harish-desktop:~$ ls
bin Desktop Documents Downloads install log Music Pictures Public snap Templates Videos
harish@harish-desktop:~$ colcon build --symlink-install

Summary: 0 packages finished [1.32s]
harish@harish-desktop:~$
```

5. Installing Visual Studio Code

We chose Visual Studio Code (VS Code) as our development environment for editing .xacro and .urdf files.

Installation steps:

1. Download the .deb package from the official VS Code website.

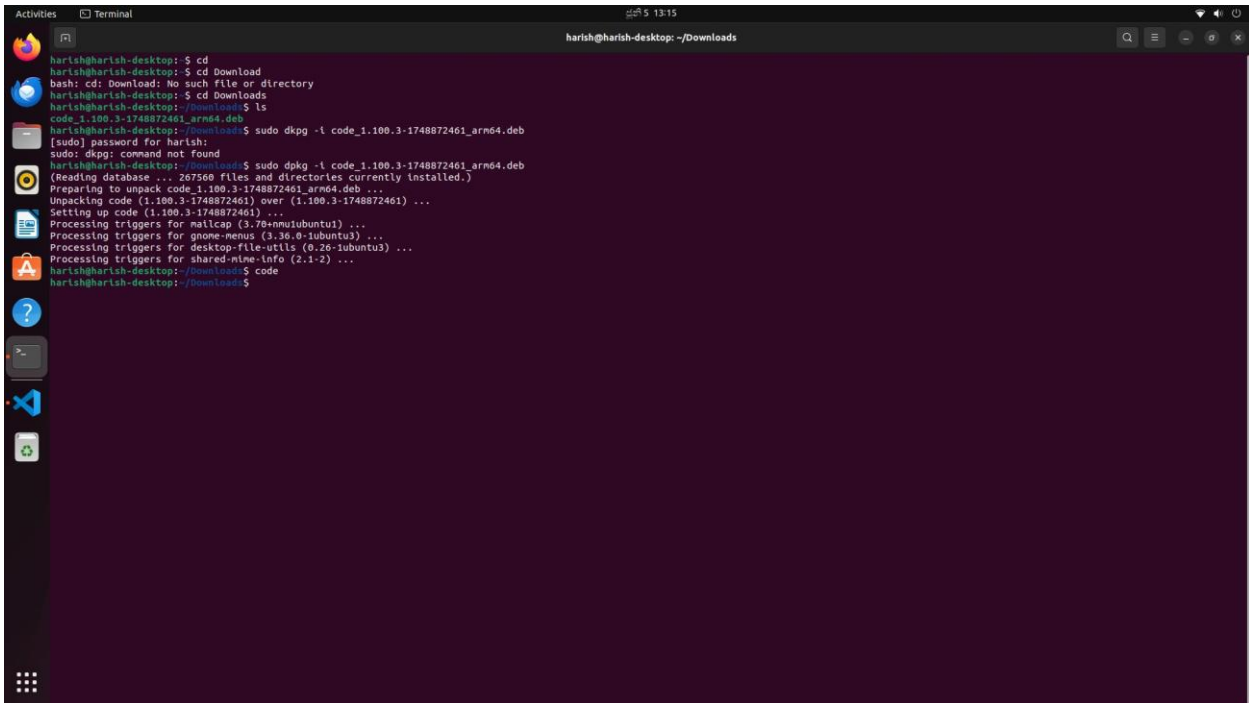
Run:

```
bash CopyEdit sudo
apt update
sudo dpkg -i <package_name>.deb
```

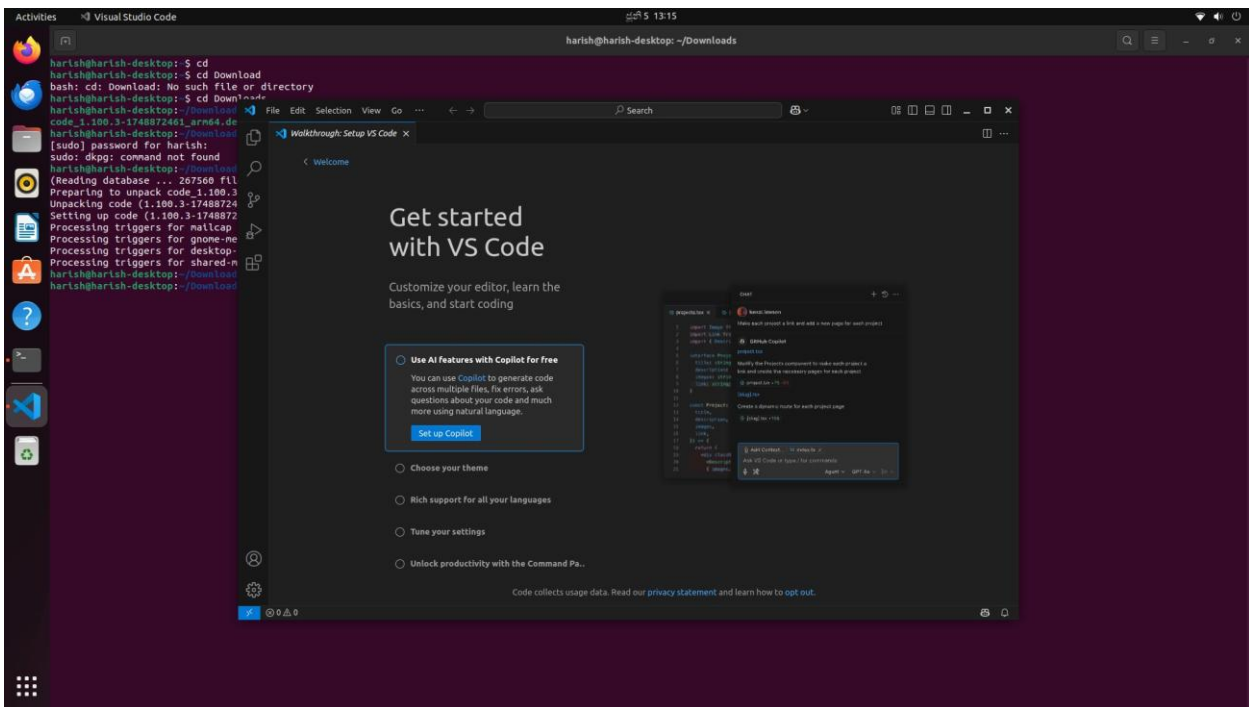
- 2.

Launch VS Code by entering:

```
bash
CopyEdit code
```



```
harish@harish-desktop: ~/Downloads$ cd
harish@harish-desktop: ~/Downloads$ cd Download
bash: cd: Download: No such file or directory
harish@harish-desktop: ~/Downloads$ cd Downloads
harish@harish-desktop: ~/Downloads$ ls
code_1.100.3-1748872461_arm64.deb
harish@harish-desktop: ~/Downloads$ sudo dpkg -i code_1.100.3-1748872461_arm64.deb
[sudo] password for harish:
sudo: dpkg: command not found
harish@harish-desktop: ~/Downloads$ sudo dpkg -i code_1.100.3-1748872461_arm64.deb
(Reading database ... 267500 files and directories currently installed.)
Preparing to unpack code_1.100.3-1748872461_arm64.deb ...
Unpacking code (1.100.3-1748872461) over (1.100.3-1748872461) ...
Setting up code (1.100.3-1748872461) ...
Processing triggers for mailcap (3.70+nmu1ubuntu1) ...
Processing triggers for gnome-menus (3.36.0-1ubuntu3) ...
Processing triggers for desktop-file-utils (0.26-1ubuntu3) ...
Processing triggers for shared-mime-info (2.1-2) ...
harish@harish-desktop: ~/Downloads$ code
harish@harish-desktop: ~/Downloads$
```



6. Creating and Sourcing a ROS2 Workspace

We created a workspace to organize our ROS2 packages and ensure environment variables are properly loaded.

Steps:

bash CopyEdit

```
mkdir -p ~/ros2_ws/src cd ~/ros2_ws
```

We then added the following line to `.bashrc` to automatically source the ROS2 environment whenever we open a terminal:

bash CopyEdit

```
echo "source /opt/ros/humble/setup.bash" >> ~/.bashrc source ~/.bashrc
```

This prevents us from having to manually source ROS2 in every new terminal session.

7. Installing Xacro and Joint State Publisher GUI

We installed Xacro to create modular, readable URDF files using macros. Xacro allows us to build maintainable and scalable robot descriptions.

We also installed Joint State Publisher GUI, which allows us to simulate and visualize joint states. This tool is critical during early development and testing phases, especially when hardware is not yet available.

Installation commands:

bash CopyEdit

```
sudo apt update  
sudo apt install ros-humble-xacro  
sudo apt install ros-humble-joint-state-publisher-gui
```

After installation, we built our workspace using:

bash CopyEdit

```
colcon build --symlink-install
```

We used the `--symlink-install` flag to avoid repeated rebuilds when modifying existing files. This is useful during frequent iterations. However, if we added any new files, we ran:

bash CopyEdit

```
colcon build
```

This command ensured the new files were properly registered in the build system.

To reflect changes in the robot description, we restarted the robot_state_publisher after each modification.

Reference: <https://articulatedrobotics.xyz/tutorials/ready-for-ros/ros-overview>

8. Error Encountered During colcon Build

After the installation and workspace setup, we attempted to launch the robot model using our Xacro-based description. However, we encountered a build error. Upon investigation, we found that the URDF/Xacro files were structured according to ROS1 conventions and were not compatible with ROS2.

Due to this error, we paused further steps such as testing robot_state_publisher, and began restructuring the files for ROS2 compatibility.

We tried few steps to resolve this problem. We use the following code to covert ROS1 to ROS2.

Step 1: Verify the URDF/Xacro file

URDF files themselves usually **don't need any changes** unless:

- You're using deprecated tags.
- You're using ROS 1-specific plugins (like Gazebo or RViz-related tags).
- Your .xacro files use old syntax (update to newer `<xacro:...>` format if necessary).

Test the xacro file:

```
bash
```

```
CopyEdit
```

```
ros2 run xacro xacro your_robot.urdf.xacro -o your_robot.urdf
```

Step 2: Convert launch files

ROS 2 uses **Python-based launch files** instead of XML (.launch.py instead of .launch).

Here's how to convert:

ROS 1 (XML style) xml

CopyEdit

```
<launch>

  <param name="robot_description" command="$(find xacro)/xacro $(find
your_package)/urdf/your_robot.urdf.xacro" />

  <node name="robot_state_publisher" pkg="robot_state_publisher" type="robot_state_publisher" />

</launch>
```

ROS 2 (Python style) python

CopyEdit

```
from launch import LaunchDescription from launch_ros.actions import Node from
ament_index_python.packages import get_package_share_directory import os

def generate_launch_description():

    urdf_file = os.path.join(    get_package_share_directory('your_package'),

                                'urdf',

                                'your_robot.urdf.xacro'

    )    return LaunchDescription([    Node(        package='robot_state_publisher',
executable='robot_state_publisher',        name='robot_state_publisher',        output='screen',
parameters=[{'robot_description': open(urdf_file).read()}]

    ),

    ])
```


Step 3: Handle dependencies

Update your `package.xml` and `CMakeLists.txt` to use ROS 2 dependencies:

In `package.xml`:

xml

CopyEdit

```
<exec_depend>robot_state_publisher</exec_depend> <exec_depend>xacro</exec_depend>
```

In `CMakeLists.txt`:

Make sure to find dependencies with:

cmake

CopyEdit

```
find_package(ament_cmake REQUIRED) find_package(xacro REQUIRED)
```

```
find_package(robot_state_publisher REQUIRED)
```

Step 4: Build and Test

1. Build your ROS 2 package:

bash CopyEdit

```
colcon build
```

2. Source your workspace:

bash CopyEdit

```
source install/setup.bash
```

3. Launch:

bash CopyEdit

```
ros2 launch your_package your_launch_file.launch.py
```

But it wont works and the error continues.